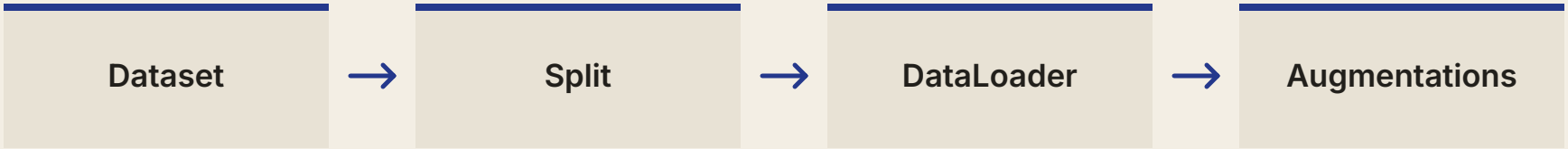
Dataset, DataLoader & Augmentations

PyTorch Image Segmentation Teaching Session

Pipeline Overview

Step-by-step data pipeline for PyTorch image segmentation models.



Sequential Execution: Raw images and masks pass from initial ingestion into structured train/val splits, get batched via PyTorch DataLoader, and undergo simultaneous spatial transformations.

Dataset Overview & Specifications

24
Labelled Training Images

4
Test Images

96
Training Annotation Rows

Image Specifications

- **Format:** RGB color channels
- **Dimensions:** Height 60 × Width 90
- **Total Items:** 28 images overall

Target Pattern Labels

- **Fish:** Marine cloud pattern
- **Flower:** Flocculent cloud pattern
- **Gravel:** Granular cloud pattern
- **Sugar:** Diffuse cloud pattern

CLOUD TYPE SEGMENTATION: ONE RGB IMAGE, FOUR BINARY MASKS

BINARY MASKS: BLACK = CLASS, WHITE = BACKGROUND



RGB CLOUD PHOTOGRAPH
(60x90 px)

FISH



FISH CLOUDS (Black)
BACKGROUND (White)

FLOWER



FLOWER CLOUDS (Black)
BACKGROUND (White)

GRAVEL



GRAVEL CLOUDS (Black)
BACKGROUND (White)

SUGAR



SUGAR CLOUDS (Black)
BACKGROUND (White)

Input vs. Target Tensors

RGB Channels [3, H, W]

- Define color intensities across standard visual channels.
- Values are scaled to **float32** representation for model input.

Target Channels [4, H, W]

- Define the binary presence of labels across mask channels.
- Background is zero across all four target channels.
- Multi-label overlap is allowed within the target tensor structure.

Dataset Indexing & Pivoting

Structuring raw annotation records into single-filename rows with ordered labels

1. RAW CSV FORMAT

Image_Label, EncodedPixels

fn1.jpg_1, 29414 2 29926...
fn1.jpg_2, 10245 8 10500...
fn1.jpg_3, NaN
fn1.jpg_4, 58210 4 58900...

4 rows per filename key

→

2. PIVOT OPERATION

read_index()

- Splits composite keys
- Validates duplicates / missing
- Groups 4 rows into 1 row

→

3. CONSOLIDATED ROW

Filename | Label_1..4

fn1.jpg | [rle1, rle2, rle3, rle4]

1 row per filename with ordered labels

Core Pipeline Operations in read_index Function

- **Key Splitting:** Parses the composite **Image_Label** string into individual filename keys and numerical class indices.
- **Data Validation:** Verifies dataset consistency by explicitly checking for missing label pairs or unintended duplicate entries.
- **Data Pivoting:** Combines four distinct CSV rows into a single structured row per filename with sequentially ordered labels.

Dataset Class Methods

__init__

Stores file paths, annotation index, target size, and optional transforms.

__len__

Returns the total number of images, which is 24.

__getitem__

Fetches a single image and decoded 4-channel mask on demand.

__getitem__ Workflow

01

Load Image & Decode Masks

Load the image and decode four original-size RLE masks.

02

Resize & Convert to Tensors

Resize the image and masks, then convert them to float32 tensors.

03

Apply Paired Transforms

Apply paired transforms if defined.

04

Return Sample Dictionary

Return a dictionary containing the image, mask, and filename.

Tensor Transposition & Scaling

Layout Transposition

HWC → CHW

Rearranges layout from standard image format **(Height, Width, Channels)** to PyTorch tensor format **(Channels, Height, Width)**.

Input: [H, W, 3]
Output: [3, H, W]

Mask Stacking

[4, H, W] Target

Four separate 2D HW masks are concatenated along a new channel dimension into a unified multi-class target tensor.

4x Mask [H, W]
→ Stack → [4, H, W]

Intensity Scaling

[0-255] → [0.0, 1.0]

Pixel values are normalized from standard 8-bit integers to floating-point values in the range [0.0, 1.0].

Verification:

Inspect outputs via visual overlay to confirm spatial alignment and scaling.

Dataset Splits Overview

Training Set

The training set updates model weights via gradients.

Validation Set

The validation set evaluates generalization during training and tunes hyperparameters without gradient updates.

Test Set

The test set serves as the final un-touched evaluation set.

Dataset Train / Validation Split

80%

19 Training Images

An 80% training split yields 19 training images from the total dataset.

20%

5 Validation Images

The remaining 20% validation split yields 5 validation images using whole-image rounding.

DATASET CONFIGURATION SUMMARY

Starting with 24 total labelled images. An 80% training split yields 19 training images. The remaining 20% validation split yields 5 validation images using whole-image rounding.

Dataset Splitting Strategy

Split image identities while keeping all four annotation rows of one image together. Never split annotation rows or augmented crops of the same photograph across training and validation sets to prevent data leakage.

> **CRITICAL RULE: NEVER SPLIT ANNOTATION ROWS AT THE IMAGE LEVEL**
>
> **All annotation rows and augmented crops derived from a single photograph must strictly remain within the same dataset split (Train or Validation) to guarantee target isolation.**

PREVENTING DATA LEAKAGE

Group-based splitting ensures that the model evaluates on entirely unseen photographic contexts, avoiding over-optimistic performance metrics caused by evaluating on crops of images present in training.

Dataset Splitting & Disjointness

- Implementation uses seeded random permutation and list slicing for the train and validation split.
- Ensures fixed membership across training epochs by fixing the random seed prior to permutation.

```
# Seeded Permutation & Dataset Slicing
g = torch.Generator().manual_seed(42)
perm = torch.randperm(len(full_dataset), generator=g).tolist()
train_ids, val_ids = perm[:n_train], perm[n_train:]
```

Verifying Strict Disjointness Across Sets

```
# Check disjointness by verifying the intersection is an empty set
intersection = set(train_ids).intersection(set(val_ids))
assert len(intersection) == 0, "Data leakage detected between splits!"
```

PyTorch DataLoader

Primary Function

Fetches dataset samples and collates individual tensors into mini-batches for processing.

batch_size

Specifies the number of samples to load in each batch for model training/evaluation.

shuffle

Set to True to have the data reshuffled at every epoch to prevent model memorization.

num_workers=0

Defines how many subprocesses to use for data loading. 0 means data is loaded in the main process.

drop_last

Set to True to drop the last incomplete batch if dataset size is not divisible by batch size.

Sample vs. Batch Tensor Shapes

Single Sample [C, H, W]

Image Shape

[3, 60, 90]

Channels = 3 (RGB), Height = 60, Width = 90

Mask Shape

[4, 60, 90]

Classes = 4 (One-Hot), Height = 60, Width = 90

Mini-Batch (Size 4) [B, C, H, W]

Image Tensor

[4, 3, 60, 90]

Batch = 4, Channels = 3, Height = 60, Width = 90

Mask Tensor

[4, 4, 60, 90]

Batch = 4, Classes = 4, Height = 60, Width = 90

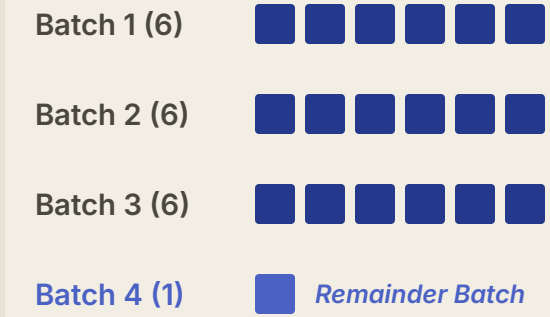
Key Takeaway: Batching prepends the Batch Dimension **B = 4** as the leading axis (0-th dimension) to both image and mask tensors.

DataLoader Behavior: drop_last

Setup: 19 training images with a **batch_size** of 6.

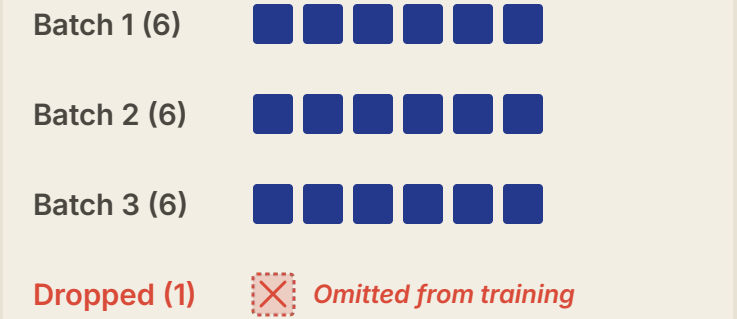
drop_last = False

If **drop_last is False**, there are **3 full batches of 6** and **1 remainder batch of 1**.



drop_last = True

If **drop_last is True**, **18 images** are used across **3 batches**, **dropping 1**.



Data Augmentation & Preprocessing

Geometric Transforms

Geometric augmentations alter spatial structure; image and mask must transform identically.

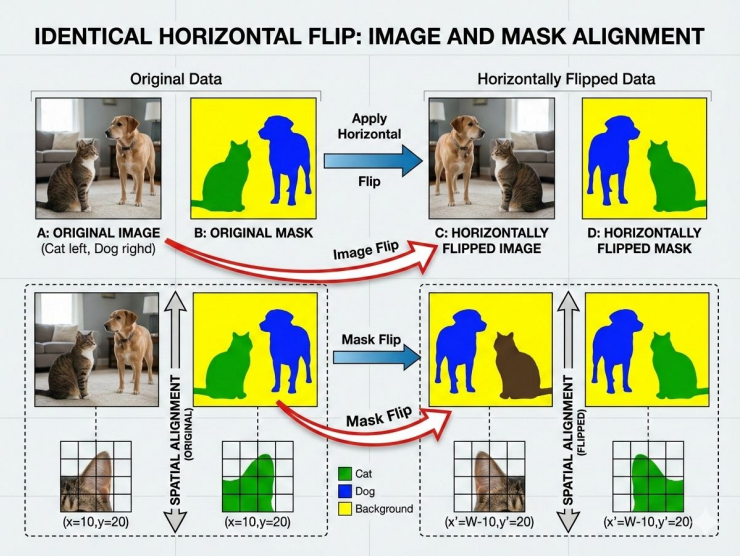
Appearance Transforms

Appearance augmentations alter color or intensity and apply only to the image.

VALIDATION & INFERENCE PREPROCESSING

Validation preprocessing uses deterministic resizing and scaling only.

AUGMENTATION ALIGNMENT



Ensuring spatial alignment requires synchronized transformations across both image data and segment masks.

Synchronized Augmentations

Flip the image and all 4 masks using identical transformation parameters.

TENSOR STRUCTURE

Width is axis -1 in CHW/4HW tensors.

INTERACTIVE LAB

Hands-on Experimentation

Explore this concept in the interactive lab:

[link removed]

Augmentation Bug: Un-Flipped Target Masks

CRITICAL AUGMENTATION BUG

Flipping the image during data augmentation while leaving target masks unflipped creates severe spatial misalignment. The transformed input image no longer corresponds geographically to its target label.

Do tensor shapes detect this bug? NO.

Image Shape: [3, 60, 90] | Target Mask Shape: [4, 60, 90]

Tensor dimensions remain completely identical before and after flipping, masking spatial misalignment from shape assertions and automated pipeline checks.

Image vs. Mask Interpolation

Brightness and contrast apply to the image only.

Bilinear Interpolation

Applied to: Continuous Images

Image interpolation uses Bilinear for smooth color transitions across continuous pixel values.

Nearest-Neighbor

Applied to: Segmentation Masks

Mask interpolation uses Nearest-Neighbor to preserve discrete binary labels and prevent invalid intermediate class values.

Mask Interpolation & Boundaries

⚠ Problem: Invalid Boundary Values

Bilinear interpolation on binary masks produces fractional boundary values.

Solution: Nearest-Neighbor Interpolation

Nearest-neighbor interpolation retains the exact binary label vocabulary of 0 and 1.

PRACTICE THIS IN THE INTERACTIVE LAB:
[link removed]

Shared Seeds & Probability Mechanics

DRAWING PROBABILITY (p)

Draw one random probability p. If p is less than the threshold, apply the flip to both image and mask.

PER-SAMPLE INDEPENDENCE

Probability p=0.5 means a 50% chance per sample, not an exact 50/50 split quota across a batch.